

Online supplement:
A Comparative Analysis of Relational Database
Access-Layer
Performance in Express.js for PostgreSQL and
MySQL

Mateusz Miotk

Faculty of Mathematics, Physics and Informatics, University of Gdańsk
`mateusz.miotk@ug.edu.pl`

This supplement carries measurement detail referenced from the main text. All tables are generated by the replication package (<https://github.com/mmioTk/express-db-access-performance>; archived at <https://doi.org/10.5281/zenodo.21356266>) from the same raw data as the main-text tables.

S1 Dispersion on MySQL

The main text reports the coefficient-of-variation table for PostgreSQL and the per-engine maxima. Table S1 gives the full per-layer, per-pattern CV on MySQL across the 25 independent replicates (maximum 6.6%, `knex`/insert).

Table S1: Coefficient of variation (%) of throughput across the 25 independent replicates, per access layer and pattern on MySQL. Low values indicate stable measurements.

Layer	<code>point_read</code>	<code>range_scan</code>	<code>deep_fetch</code>	<code>aggregation</code>	<code>write</code>
<code>mysql2</code>	3.2	2.7	2.2	1.8	6.6
<code>knex</code>	2.4	3.0	2.2	2.6	6.6
<code>drizzle</code>	3.0	2.2	2.4	2.2	5.2
<code>prisma</code>	4.2	2.0	1.0	4.0	5.7
<code>sequelize</code>	2.2	2.0	1.9	2.1	5.9
<code>typeorm</code>	2.5	1.9	3.0	2.5	6.0
<code>objection</code>	3.3	3.4	2.5	2.5	5.9
<code>mikroorm</code>	3.5	2.9	3.0	1.9	4.9

S2 Idiomatic round-trip counts

Table S2 reports, for each layer, the number of SQL statements the idiomatic deep fetch issues per request, captured from server-side statement logs. The counts motivate the main text’s observation that round-trip count alone does not determine throughput: the two layers issuing four statements (Prisma, Objection) sit at opposite ends of the throughput ladder.

Table S2: Number of SQL round trips each access layer issues per request on PostgreSQL, captured from server-side statement logging. The native drivers, Knex, and Drizzle issue a two-query deep fetch; the data-mapper ORMs’ eager-loading strategies choose their own counts. Full generated SQL and EXPLAIN plans are in the replication package.

Layer	Point read	Range scan	Deep fetch	Aggregation	Insert
pg	1	1	2	1	1
knex	1	1	2	1	1
drizzle	1	1	2	1	1
prisma	1	1	4	1	1
sequelize	1	1	1	1	1
typeorm	1	1	2	1	2
objection	1	1	4	1	1
mikroorm	1	1	1	1	1

S3 Write throughput under default versus relaxed durability

Table S3 gives the per-layer insert throughput under each engine’s default durability configuration (the paper’s primary write regime) and under the symmetric relaxed regime (secondary, 10 replicates), with the relaxed÷default ratio quoted in the main text.

S4 Equal-CPU-budget control

Table S4 gives the deep-fetch throughput with the server confined to 1, 2, or 4 cores (database and load generator pinned to disjoint cores); all nine values are quoted in the main text.

S5 Application-tier versus end-to-end CPU efficiency

Table S5 reports, per layer on the deep/nested fetch, the median application-process-tree CPU and database-server CPU, the combined core-count, and

Table S3: Insert throughput (req/s) under the default durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`; median of 25 replicates) and under the relaxed regime (all of the above off; median of 10), with the relaxed÷default ratio. At 50 concurrent connections, group commit amortizes the per-commit flushes: relaxing durability buys at most 14% on PostgreSQL (and $\leq 4\%$ on the layer-bound ORMs) and 19–24% on MySQL.

Layer	PostgreSQL			MySQL		
	default	relaxed	ratio	default	relaxed	ratio
pg	6064	6895	1.14	–	–	–
mysql2	–	–	–	809	966	1.19
pg-tuned	6280	7150	1.14	–	–	–
mysql2-tuned	–	–	–	808	972	1.20
knex	4693	5212	1.11	795	948	1.19
drizzle	4230	4675	1.11	796	950	1.19
prisma	5465	5675	1.04	714	879	1.23
sequelize	2668	2653	0.99	772	925	1.20
typeorm	2595	2675	1.03	732	886	1.21
objection	3926	4245	1.08	787	944	1.20
mikroorm	1571	1607	1.02	663	822	1.24

Table S4: Deep-fetch throughput (req/s, PostgreSQL, median of 3) with the server process confined by `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores). `pg` reaches full throughput on a single core; Prisma requires roughly four to approach it, quantifying the CPU subsidy behind its near-native throughput.

Layer	1 core	2 cores	4 cores
pg	3663	3277	3700
prisma	896	1726	3025
mikroorm	463	577	542

throughput per application-CPU-second and per combined (application+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database: `pg` leads application-tier efficiency by roughly $5\times$ over Prisma (3,741 vs. 740 req per app-CPU-s) but only $1.4\times$ end-to-end (813 vs. 596 req per combined-CPU-s), because it pushes about 3.6 database cores of work that the application-only metric ignores.

Table S5: Application-tier versus end-to-end CPU efficiency on the deep/nested fetch (PostgreSQL): median CPU (% of one logical core) of the Node.js process tree and of the database server, the combined core-count, and throughput per application-CPU-second and per combined (app+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database; the combined figure is the end-to-end view. Load-generator CPU is excluded.

Layer	app CPU%	db CPU%	comb. cores	req/app-CPU-s	req/comb-CPU-s
pg	100	360	4.60	3741	813
knex	100	245	3.45	2491	722
drizzle	100	205	3.05	2080	682
prisma	498	120	6.18	740	596
sequelize	100	50	1.50	989	659
typeorm	105	120	2.25	1264	590
objection	100	50	1.50	1037	691
mikroorm	100	45	1.45	571	394

S6 Coordinated-omission-corrected open-loop sweep

Table S6 gives the full constant-arrival sweep (PostgreSQL, five layers, offered rates 250–4,000 requests/s) behind the open-loop paragraph of the main text: achieved throughput, corrected latency percentiles measured from each request’s *intended* send time, and timeout counts (10 s cap, clipped into the distribution).

S7 Pool-size sensitivity

The primary campaign fixes every layer’s connection pool at 10 to isolate the access layer from pool tuning. Table S7 varies the pool from 1 to 50 for a representative layer of each tier on the deep fetch (PostgreSQL, 50 connections). The three-band ordering (pg/Prisma, then Knex, then MikroORM) is preserved at every pool size, and each layer’s throughput saturates at a pool well below 50, so the fixed-pool choice does not drive the ranking; a very small pool (1–2) is the one regime that compresses the differences, because every layer then queues on connection acquisition.

Table S6: Open-loop tail latency, corrected for coordinated omission: p99 (ms) on the deep/nested fetch (PostgreSQL) under a constant offered request rate, with every latency measured from the request’s intended start time and timed-out requests included at their clipped value. [†]marks cells that did not sustain the offered rate (achieved<95% or any timeouts).

Layer	250/s	500/s	1000/s	2000/s	4000/s
pg	2	2	2	2	1201 [†]
prisma	3	2	2	4	2110 [†]
knex	2	2	2	33	9420 [†]
sequelize	2	3	41	10757 [†]	10944 [†]
mikroorm	3	15	11220 [†]	11399 [†]	11878 [†]

Table S7: Deep-fetch throughput (req/s, PostgreSQL, 50 connections, median of 3) as the connection pool varies from 1 to 50 for a representative layer of each tier. The primary campaign fixes the pool at 10; the three-band ordering (native and query-engine ORM on top, the query builder in the middle, the data-mapper ORM below) is preserved at every pool size, and each layer saturates at a pool well below 50, so the fixed-pool choice does not drive the ranking.

Layer	1	2	5	10	20	50
pg	895	1732	3203	3672	3764	3697
knex	750	1423	2455	2499	2648	2577
prisma	949	1670	3446	3351	3651	3565
mikroorm	535	539	548	570	584	587

S8 Transactional multi-statement write

Beyond the single-row insert of the primary matrix, Table S8 reports a transactional write, `POST /threads`, which inserts a post and five comments in one transaction through each layer’s transaction facility, for a representative layer of each tier on PostgreSQL under default durability. The full layer $\times$ engine matrix for this pattern is left to future work. Because the transaction commits once, its per-commit `fsync` dominates and the layers cluster much more tightly than the between-layer spread on reads; the ordering tracks the single-row insert, so the write conclusions extend to a multi-statement transactional write.

Table S8: Transactional write throughput (req/s) and tail latency (p99, ms) for `POST /threads`, which inserts a post and five comments in a single transaction, on PostgreSQL under default durability (50 connections, median of 5). A representative layer of each tier is shown; the full matrix is left to future work. Between-layer spread is $3.3\times$ (Prisma 2412 down to MikroORM 740), well below the $6.55\times$ read spread, and the ordering tracks the single-row insert, so the write conclusions extend to a multi-statement transactional write.

Layer	req/s	p99 (ms)
pg	2347	44
knex	2029	49
prisma	2412	41
typeorm	1790	48
mikroorm	740	90

S9 Dispersion on PostgreSQL

Table S9 gives the full per-layer, per-pattern coefficient of variation of throughput on PostgreSQL across the 25 independent replicates (maximum 7.6%, `pg/insert`), the companion to the MySQL table in Section S1.

S10 Resource footprint

Table S10 gives the per-layer application-process CPU (median % of one logical core) and peak resident memory on the deep/nested fetch (PostgreSQL), the detail behind the CPU trade-off discussed in the main text (Prisma $\approx 498\%$, the other layers $\approx 100\%$; peak RSS 204–271 MB).

Table S9: Coefficient of variation (%) of throughput across the 25 independent replicates, per access layer and pattern on PostgreSQL. Low values indicate stable measurements.

Layer	point_read	range_scan	deep_fetch	aggregation	write
pg	4.3	2.9	2.6	1.9	7.6
knex	2.6	3.1	3.2	3.8	5.5
drizzle	2.5	2.6	2.9	2.8	3.6
prisma	3.6	3.4	3.6	4.3	4.3
sequelize	3.1	2.5	2.2	1.7	3.8
typeorm	2.8	2.6	2.0	2.6	3.8
objection	3.9	3.0	2.9	2.7	3.4
mikroorm	2.7	2.4	2.9	2.9	3.7

Table S10: Server-process resource footprint on the deep/nested fetch (PostgreSQL): median CPU utilization (% of one core) and peak resident memory (MB) of the Node.js process under load. The load generator and database run in separate processes.

Layer	Category	CPU (%)	RSS (MB)
pg	native-driver	100	205
knex	query-builder	100	204
drizzle	orm-lightweight	100	250
prisma	orm	498	266
sequelize	orm	100	248
typeorm	orm	105	271
objection	orm	100	211
mikroorm	orm	100	255

S11 Pinned versions

Table S11 lists the pinned versions of every evaluated access layer and tool, recorded from the logfile on the measurement date.

Table S11: Pinned versions of the evaluated access layers and tooling.

Layer / tool	Version	Layer / tool	Version
pg	8.22.0	sequelize	6.37.8
mysql2	3.22.5	typeorm	0.3.30
knex	3.3.0	objection	3.1.5
drizzle-orm	0.36.4	@mikro-orm/core	6.6.15
@prisma/client	5.22.0	express	4.22.2
autocannon	7.15.0	PostgreSQL / MySQL	18.4 / 9.7.1